

Why the Slider Error Repeated

A beginner-friendly explanation of the size check, pointer events, and the fixes.

The short answer: the slider checked the size correctly, but after showing the error it continued starting the drag. Also, the size button could look selected without updating the real order data. Those two things made the page feel stuck.

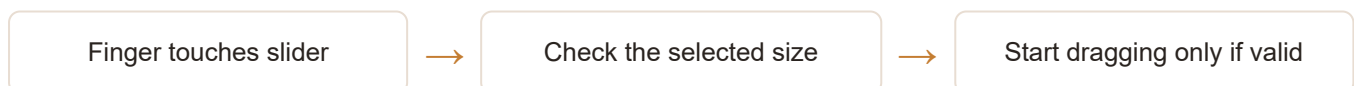
1. Where the size is stored

Your order begins with an empty size:

```
size: {  
  name: "",  
  Price: 0  
}
```

An empty name means that the user has not chosen a size. After a size click, `SizeChooosed(event)` should change it to a real value such as `"Small"`.

2. What happens when the user slides



The check lives in `Ticking_SoundPlayer()`:

```
if (OrderObject.option.size.name == "") {  
  window.alert("Select first");  
  return false;  
}  
  
return true;
```

`false` means “the check failed.” `true` means “it is safe to continue.”

3. The original mistake

The slider used to call the check, but ignored its answer:

```
Dragging = true;
Ticking_SoundPlayer();
SliderButton.setPointerCapture(event.pointerId);
```

Calling a function does not automatically stop the current function. Even when the alert appeared, the next lines still ran. `setPointerCapture()` tells the browser to keep sending pointer events to the slider. On a phone, that can make later taps feel like the slider still owns the touch.

4. The important fix

```
if (!Ticking_SoundPlayer()) return;

Dragging = true;
SliderButton.setPointerCapture(event.pointerId);
```

Read this as: “If the check does *not* return true, leave this function now.” The drag only starts after a valid size has been confirmed.

5. The size-click mistake

The click counter used to be allowed to stop the handler before the order was updated:

```
HandleClicks(Size);
if (Size.dataset.Clicked == "true") return;
SizeChooosed(event);
```

This created two different truths:

- The button could look selected through `dataset.Clicked`.
- The real order could still have `size.name == ""`.

The slider checks the real order object, not the button’s appearance.

6. The size-click fix

```
HandleClicks(Size);
SizeChooosed(event);
```

Now every real size-button click updates:

```
OrderObject.option.size.name
OrderObject.option.size.Price
```

The visual click tracking can still run, but it cannot prevent the important data update.

7. A small safety check

```
let Size = event.target.closest("button");  
if (!Size) return;
```

`closest("button")` can return nothing if the click did not come from a button. The safety check prevents code from trying to use a missing button.

Remember this debugging rule: when a function returns a success or failure value, the caller must use that value. Write `if (!check()) return;`, not just `check();`. Also trust the real data object more than a visual class, attribute, or style.

Files involved: `java-buying.js`, `OrderBuilder/Ui/TickingSoundRepeat.js`, and `OrderBuilder/Ui/SlideToOrder.js`.